



JC4002 Knowledge Representation

Day 5: Prolog III

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 5

Prolog III

1. Arithmetic
2. Loops
3. Lists
4. Built-in Predicates

Arithmetic in Prolog

- Functor **is/2**: number (or variable) **is** math expression
- Equality functor **==** (remember that **=** is for unification!)
- Inequality functor **=\=** (**\=** is for two things not unified!)

math notation	Prolog notation
$6 + 2 = 8$	8 is 6 + 2. or 8 is +(6, 2).
$6 - 2 = 4$	4 is 6 - 2.
$6 - 8 = -2$	-2 is 6 - 8.
$6 * 2 = 12$	12 is 6 * 2.
$6/2 = 3$	3 is 6/2.
$7/2 = X$	X is 7/2.
1 remainder of 7/2	1 is mod(7, 2).
$2 + 2 = 2 * 2$	$2 + 2 == 2 * 2.$
$2 + 2 \neq 2 * 2$	$2 + 2 =\!= 2 * 2.$
$A = (X + Y) * Z$	add_and_multiply(A,X,Y,Z) :- A is (X + Y) * Z.

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$4 \text{ is } X.$

$1 + 2 \text{ is } 1 + 2.$

$\text{is}(X, +(1, 2))$

$2 = \backslash = 1 * 2.$

$*(1, (2 + 3)) = 1 * (+(2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$4 \text{ is } X.$

$1 + 2 \text{ is } 1 + 2.$

$\text{is}(X, +(1, 2))$

$2 = \backslash = 1 * 2.$

$*(1, (2 + 3)) = 1 * (+ (2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

$1 + 2 \text{ is } 1 + 2.$

$\text{is}(X, +(1, 2))$

$2 = \backslash = 1 * 2.$

$*(1, (2 + 3)) = 1 * (+(2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

Error, open variable is not a math expression

$1 + 2 \text{ is } 1 + 2.$

$\text{is}(X, +(1, 2))$

$2 = \backslash = 1 * 2.$

$*(1, (2 + 3)) = 1 * (+(2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

Error, open variable is not a math expression
false.

$1 + 2 \text{ is } 1 + 2.$

$\text{is}(X, +(1, 2))$

$2 = \backslash = 1 * 2.$

$*(1, (2 + 3)) = 1 * (+ (2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

Error, open variable is not a math expression

$1 + 2 \text{ is } 1 + 2.$

false.

$\text{is}(X, +(1, 2))$

$X = 3.$

$2 \neq 1 * 2.$

$*(1, (2 + 3)) = 1 * (+(2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

Error, open variable is not a math expression

$1 + 2 \text{ is } 1 + 2.$

false.

$\text{is}(X, +(1, 2))$

$X = 3.$

$2 \neq 1 * 2.$

false.

$*(1, (2 + 3)) = 1 * (+(2, 3)).$

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Arithmetic Examples

What is the Prolog answer for the following queries?

$X = 3 * 4.$

$X = 3 * 4.$

$X \text{ is } 3 * 4.$

$X = 12.$

$4 \text{ is } X.$

Error, open variable is not a math expression

$1 + 2 \text{ is } 1 + 2.$

false.

$\text{is}(X, +(1, 2))$

$X = 3.$

$2 \neq 1 * 2.$

false.

$*(1, (2 + 3)) = 1 * (+(2, 3)).$ true.

More arithmetic functions available here:

<https://www.swi-prolog.org/pldoc/man?section=funcsummary>

Day 5

Prolog III

1. Arithmetic
- 2. Loops**
3. Lists
4. Built-in Predicates

Loops in Prolog

- ❑ No traditional loops (except for built-in predicate forall)
- ❑ We can obtain recursive calls by using recursive predicates
- ❑ For example:
 - descendant(X,Y) :- child(X,Y).
 - descendant(X,Y) :- child(X,Z), descendant(Z,Y).
- ❑ Why not: descendant(X,Y) :- child(X,Z), child(Z,Y). ?

Loops Examples

□ Considering the following Prolog KB:

- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z), descendant(Z,Y).`

□ How would Prolog answer the query:

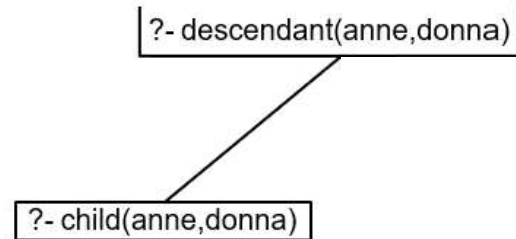
- `?- descendant(anne,donna).`

Search Tree

`?- descendant(anne,donna)`

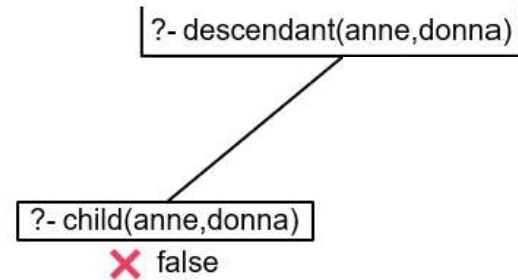
- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z),
descendant(Z,Y).`

Search Tree



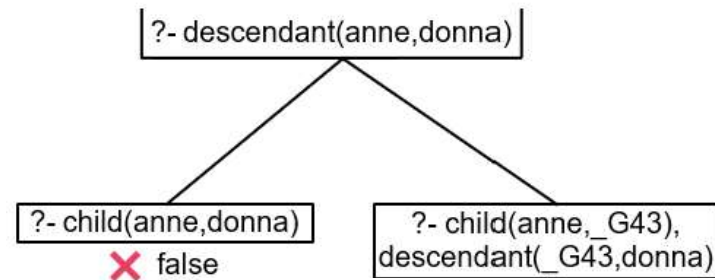
- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z),
descendant(Z,Y).`

Search Tree



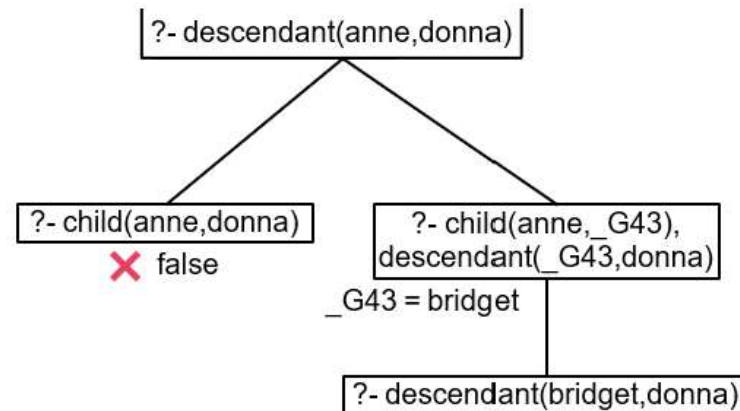
- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z),
descendant(Z,Y).`

Search Tree



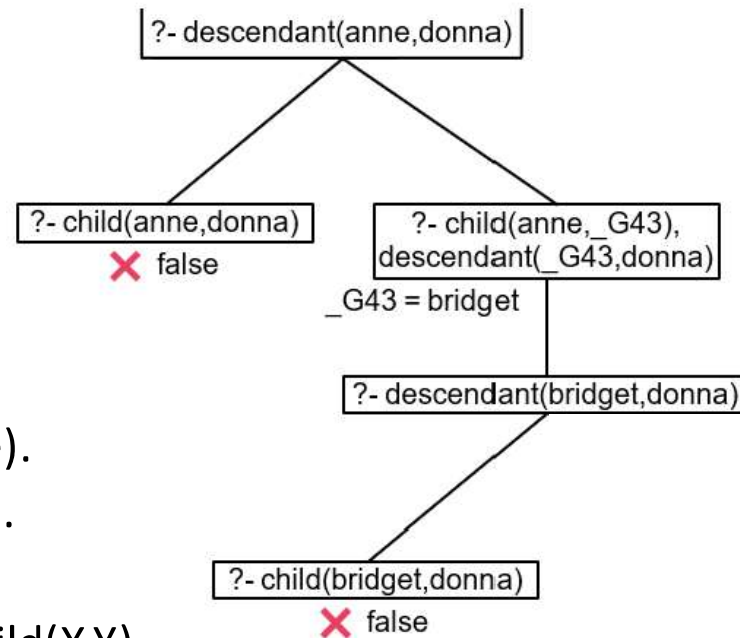
- child(anne,bridget).
- child(bridget,caroline).
- child(caroline,donna).
- child(donna,emily).
- descendant(X,Y) :- child(X,Y).
- descendant(X,Y) :- child(X,Z), descendant(Z,Y).

Search Tree



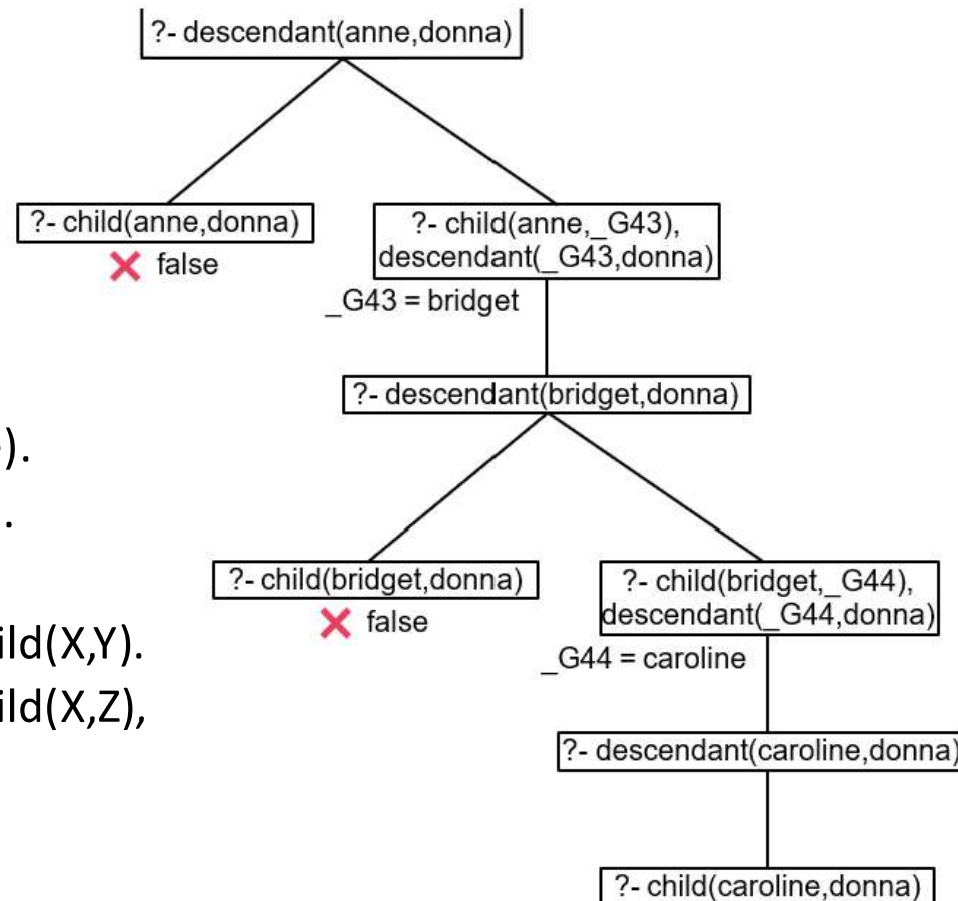
- child(anne,bridget).
- child(bridget,caroline).
- child(caroline,donna).
- child(donna,emily).
- descendant(X,Y) :- child(X,Y).
- descendant(X,Y) :- child(X,Z), descendant(Z,Y).

Search Tree



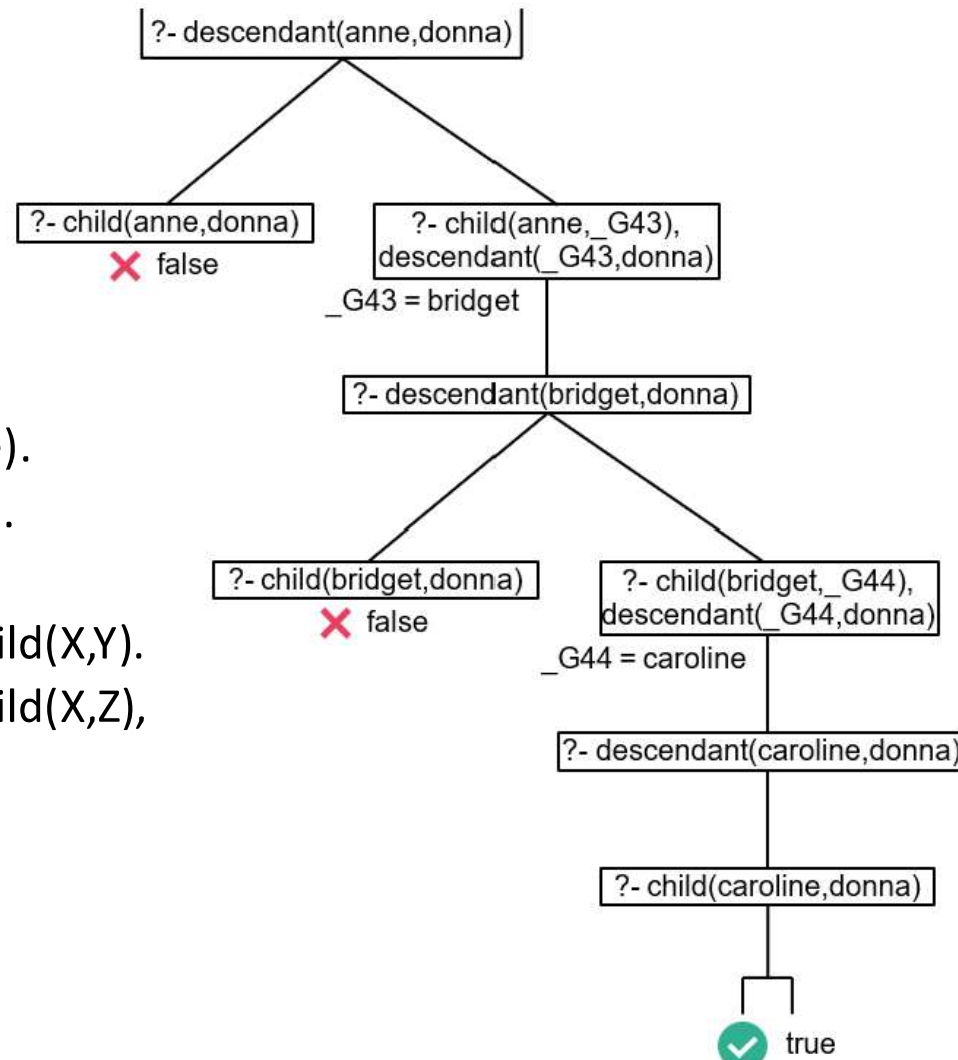
- child(anne,bridget).
- child(bridget,caroline).
- child(caroline,donna).
- child(donna,emily).
- descendant(X,Y) :- child(X,Y).
- descendant(X,Y) :- child(X,Z), descendant(Z,Y).

Search Tree



- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z),
descendant(Z,Y).`

Search Tree



- `child(anne,bridget).`
- `child(bridget,caroline).`
- `child(caroline,donna).`
- `child(donna,emily).`
- `descendant(X,Y) :- child(X,Y).`
- `descendant(X,Y) :- child(X,Z), descendant(Z,Y).`

Day 5

Prolog III

1. Arithmetic
2. Loops
- 3. Lists**
4. Built-in Predicates

Lists

- Lists in Prolog begin with “[” and end with “]”.
- Values are separated by “,”.
- Prolog provides a nice notation for easily accessing the first element of a list.
 - | ?- [Head | Rest] = [a, b, c, d].
 - | ?- Head = a, Rest = [b, c, d].
- You can also get more than one item at the head of the list.
 - | ?- [First, Second | Rest] = [a, b, c, d].
 - | ?- First = a, Second = b, Rest = [c, d].
- With this flexibility, we can write a function called `member(X, List)` that succeeds when `X` is somewhere on the list `List`.
 - `member(X, [X | _]).` % base case
 - `member(X, [Head | List]) :- member(X, List).` % recursive case

Manipulating Lists

- `member(X, L).`
 - `member(3, [1,2,3]).`
 - `true.`
- `append(L1, L2, LR).`
 - `append([1,2], [3], LR).`
 - `LR = [1,2,3].`
- `nth0(I, L, Element).` (nth1 version)
 - `nth0(X, [1,3], 1).`
 - `X = 0; false.`
- `nth0(I, LR, Element, L).` (nth1 version)
 - `nth0(2, LR, 4, [1,3]).`
 - `LR = [1,3,4].`
- `delete(L, Element, LR).`
 - `delete([1,2,3], 3, LR).`
 - `LR = [1,2].`
- `length(L, N).`
 - `length([1,2,3], N).`
 - `N = 3.`
- As “length” is built-in, let’s create our own
 - `mylen([], 0).`
 - `mylen([_ | X], L) :- mylen(X, R),`

`L is 1 + R.`
- Can you create simple statistics from lists?
 - Sum
 - Mean
 - Min
 - Max

Day 5

Prolog III

1. Arithmetic
2. Loops
3. Lists
4. **Built-in Predicates**

Cut Operator (!)

- ❑ The ! (cut) operator is a very useful operator for stopping backtracking
- ❑ Can be used in the body of a rule
- ❑ It succeeds the first time when called, but then fails upon any backtracking attempt.

Without Cut

Program :	Query Prompt :
s(m).	?- solve(X, Y, Z).
s(d).	X = 3, Y = 4, Z = 5 ;
t(k).	X = m, Y = k, Z = c ;
t(i).	X = m, Y = k, Z = j ;
d(c).	X = m, Y = i, Z = c ;
d(j).	X = m, Y = i, Z = j ;
	X = d, Y = k, Z = c ;
solve(3,4,5).	X = d, Y = k, Z = j ;
solve(X,Y,Z) :- s(X), t(Y), d(Z).	X = d, Y = i, Z = c ;
solve(1,2,3).	X = d, Y = i, Z = j ;
	X = 1, Y = 2, Z = 3.

With Cut

Program :	Query Prompt :
s(m).	?- solve(X,Y,Z).
s(d).	X = 3, Y = 4, Z = 5 ;
t(k).	X = m, Y = k, Z = c ;
t(i).	X = m, Y = k, Z = j ;
d(c).	X = m, Y = i, Z = c ;
d(j).	X = m, Y = i, Z = j.
solve(3,4,5).	
solve(X,Y,Z) :- s(X), !,	
t(Y),d(Z).	
solve(1,2,3).	

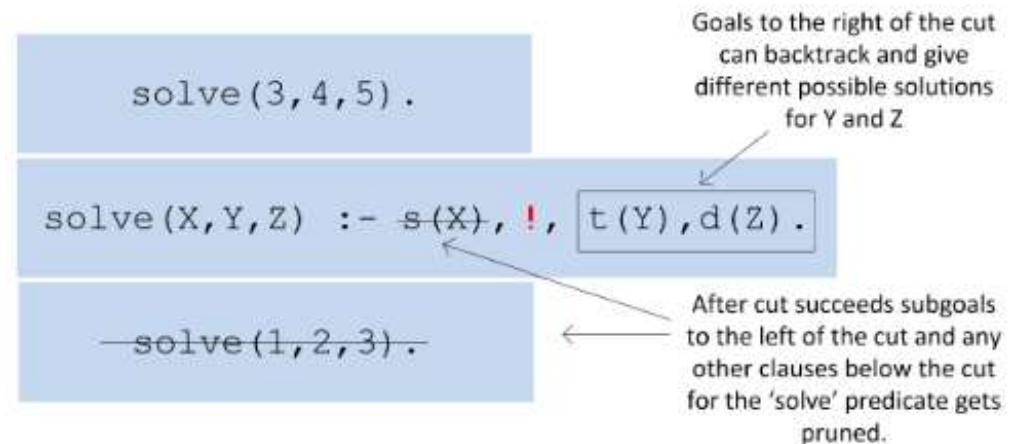
Cut Operator (!)

- It succeeds the first time when called, but then fails upon any backtracking attempt.

Program :

```
s(m).
s(d).
t(k).
t(i).
d(c).
d(j).

solve(3,4,5).
solve(X,Y,Z) :- s(X), !, t(Y), d(Z).
solve(1,2,3).
```



Query Prompt

```
?- solve(X,Y,Z).
X = 3, Y = 4, Z = 5 ;
X = m, Y = k, Z = c ;
X = m, Y = k, Z = j ;
X = m, Y = i, Z = c ;
X = m, Y = i, Z = j.
```

Goals to the right of the cut can backtrack i.e. `t(Y), d(Z)` backtracks and gives different possible solutions for Y and Z

Goals to the left of the cut no longer backtrack i.e. value for X in `s(X)` remains the same after cut executed

Activity 1

In pairs, discuss the following question (5 minutes) and then share your answers with the rest of your colleagues (5 minutes):

“Which program has a safe cut? Can you give an example for why the other one is not safe.”

Program 1

$\text{max}(X, Y, Y) \text{ :- } X \leq Y, !.$

$\text{max}(X, Y, X) \text{ :- } X > Y.$

Program 2

$\text{max}(X, Y, Y) \text{ :- } X \leq Y, !.$

$\text{max}(X, Y, X).$

If-then-else

- ❑ Prolog has a built-in predicate which allows to express if-then-else condition.
- ❑ *If A then B else C* is written as

$(A \rightarrow B; C)$

- ❑ This means:
 - ❑ Try and prove *A*. If you can, go on to prove *B* and ignore *C*.
 - ❑ If proving *A* fails, go on to prove *C* and ignoring *B*.
- ❑ The *max* predicate can be constructed with *if-then-else*.
 - ❑ $Max(X, Y, Z) :- (X \leq Y \rightarrow Z = Y; Z = X).$

Other Useful Built-in Predicates

functor	description	example
write/1	prints something on screen	write('Hello world!').
assert/1	adds a clause to the KB	assert(at(X,Y)).
retract/1	removes a clause from the KB	retract(at(X,Y)).
retractall/1	removes all facts that match	:- retractall(at(_,_)).
dynamic/1	predicate may change at runtime	:- dynamic at/2.
>, <, >=, =<	comparison	2 > 1.
fail/0	always false	fail.
halt/0	exits prolog	halt.

Full list available at:

<https://www.swi-prolog.org/pldoc/man?section=predsummary>

Tips Debugging

- The **make.** command can rebuild the Prolog program that is loaded in the interpreter (**only works if the file was modified**)
- If you want to reset your KB, you have to ensure that you have a **retractall** for every dynamic fact in your KB and then use **[program]**. (equivalent to **consult(program).**) where “program” is the name of your program (may have to include full path) **IF YOU ARE UNSURE, THE BEST WAY IS TO CLOSE THE INTERPRETER AND THEN OPEN IT AGAIN**
- Print your KB constantly while testing to make sure things are as you want and take advantage of the interpreter to debug your code
- If you want more in-depth debugging, check out [this tutorial](#)

Summary

- A deeper look into Prolog
- Arithmetic symbols
- Loops and recursive predicates
- List manipulation
- Cut operator and others

For today practical:

- You should have installed SWI-Prolog to your computer.